

— Objetivos —

Os alunos deverão desenvolver as seguintes competências e habilidades:

- Compreender o conceito de strings em C e aprender a manipular vetores de caracteres
- Implementar operações básicas sobre strings em C

Exercícios

Exercícios sobre Strings:

1) Fazer um programa que leia uma letra (L) e um número (N), a seguir gere uma *string* contendo N letras L.

2) Fazer um programa que leia uma *string* e a partir desta gere uma nova duplicando cada caracter da *string* original. Escreva a nova *string*. Por exemplo, para as strings abaixo, o programa deverá escrever:

```
"OI" => "OOII"  
"PROVA 1" => "PPRROOVVAA 11".
```

3) Fazer um programa que leia uma *string* e a partir desta gere uma nova contendo um espaço em branco entre cada caracter da *string* original. Escreva a nova *string*. Por exemplo, para a *string*

```
"LIVRO DE C"
```

o programa deverá gerar a *string*:

```
"L I V R O D E C"
```

4) Escreva uma função em C com o seguinte protótipo

```
int palindromo(const char str[])
```

A função deverá retornar 1 caso a *string* for palíndromo e 0 caso contrário. São exemplos de *strings* palíndromos:

```
ARARA  
ovo
```

5) Idem ao anterior, mas sem distinção de maiúsculas e minúsculas. Por exemplo, a função deverá retornar 1 para as *strings*:

```
Arara  
Ovo  
OsSo
```

6) Escreva uma função em C com o seguinte protótipo

```
int str2int(const char str[])
```

que receba uma *string* com um número válido, após converta o número lido para decimal, armazenando-o em uma variável inteira e a retorne.

7) Escreva uma função em C com o seguinte protótipo

```
int int2str(int n, char str[])
```

que receba um número inteiro positivo e armazene os dígitos do mesmo em uma *string* recebida por parâmetro.

8) Escreva uma função em C com o seguinte protótipo

```
int numchrtimes(const char str1[])
```

A função recebe um *string* e retorna o número de ocorrências de caracteres numéricos no *string*. Por exemplo, a chamada

```
numchrtimes("alfal bravo2 charlie 5")
```

retorna 3.

9) Escreva uma função em C com o seguinte protótipo

```
int chrtimes(const char str1[], const char ch)
```

A função recebe um *string* e um caractere e retorna o número de ocorrências da letra correspondente ao caractere no *string*. Por exemplo, a chamada

```
chrtimes("paranapanema", 'a')
```

retorna 5 (observe que as letras maiúsculas e minúsculas tem códigos ASCII diferenciados).

10) O algoritmo das pontas de criptografia recebe uma *string* como argumento e produz uma outra *string* como resultado, e pode ser descrito da seguinte forma: enquanto a *string* de entrada contiver caracteres, remova o primeiro e o último caracteres da *string* de entrada e os coloque na *string* de saída. Dessa forma, se a *string* “Programação em Java” for entrada no algoritmo, este mostrará como saída a *string* “ParvoagJr ammea çoã”. Desenvolva um programa para decodificar uma *string* usando esse algoritmo.

11) Escreva uma função em C com o seguinte protótipo:

```
void int2bin(int n, char str[])
```

A função recebe um número inteiro sem sinal e coloca em um *string* a representação do número em binário. A obtenção da representação em binário de um número é feita através de divisões (inteira) sucessivas do número por 2. O resto de cada divisão fornece os dígitos binários.

12) Escreva a função

```
int strin(const char str1[],const char str2[])
```

A função verifica se *str1* está contido em *str2*. Em caso afirmativo, a função retorna 1, em caso negativo, 0. Por exemplo, as chamadas

```
strin("ASTRO", "ASTRONAUTA");  
strin("astro", "mastro");
```

retorna 1, mas a chamada

```
strin("carro", "caravana");
```

retorna 0.

13) Faça programa em C que recebe um *string* de até 1000 caracteres contendo um texto no qual as palavras são separadas pelo caractere '#' que é seguido por um valor que identifica o número de espaços em brancos a serem inseridos pela função. O programa deverá substituir o caractere '#' e o valor pelos número de espaços definidos. Por exemplo, se o *string* original contém:

```
Isto#4é#5uma#6pequena#1frase.
```

após a chamada o string ficará:

```
Isto    é        uma        pequena frase.
```

Solução dos Exercícios sobre Strings

Exercícios Complementares:

1) Faça um programa que leia uma string e determine o percentual de frequência de vogais no texto. Por exemplo, para a string

```
programacaodecomputadoresII
```

o percentual de vogais é 48.1481%. Considere vogais em maiúsculo ou minúsculo e escreva o resultado usando uma precisão de quatro casas decimais.

2) Faça uma função, chamada `somaDigitos()` que receba uma string como parâmetro e retorne a soma dos dígitos que estão em uma string. Caracteres que não são dígitos devem ser desconsiderados. A seguir, alguns exemplos de chamadas da função e o respectivo valor a ser retornado pela função são apresentados:

```
somaDigitos("abcde") deve retornar 0
somaDigitos("1234") deve retornar 10
somaDigitos("a36]8") deve retornar 17
somaDigitos("") deve retornar 0
```

3) Implemente um programa que leia uma string com o nome de um indivíduo e gere uma outra string formada apenas pelo sobrenome.

Exemplos:

João Paulo da Silva	Silva
Luis Fernando Tavares	Tavares

4) Faça um programa que leia uma string deverá escrever o maior número existente na string, e a posição onde este começa. Os números estão separados pelo caracter '#'. Por exemplo, para a string

```
10#20#191#7#34
```

o programa deve escrever

```
Numero: 191
```

5) Escreva uma função em C com o seguinte protótipo

```
int hex2int(const char str[])
```

que receba uma *string* com um número em hexadecimal, após converta o número lido para decimal, armazenando-o em uma variável inteira e a retorne.

6) Escreva uma função em C com o seguinte protótipo

```
int int2hex(int n, char str[])
```

que receba um número inteiro positivo e armazene o número convertido para hexadecimal em uma *string* recebida por parâmetro.

Solução dos Exercícios sobre Strings (Complementares)

Strings - Trabalho Discente Efetivo - TDE

1) Faça um programa em linguagem C que leia uma string e verifique se essa corresponde a um endereço de IP (Internet Protocol) válido. Um IP corresponde a uma sequência de 4 valores numéricos compreendidos no intervalo [0-255] e separados entre si por um carácter “.”.

String lida	Saída Gerada
200.135.80.9	IP válido
192.168.1.1	IP válido
8.35.67.74	IP válido
257.32.4.5	IP inválido
85.345.1.2	IP inválido
1.2.a.4	IP inválido
9.8b.234.5	IP inválido
.168.0.255	IP inválido
192.168.0.255.2	IP inválido
192.168.0.255.	IP inválido
8.35..74	IP inválido

OBS: não podem ser utilizadas funções de manipulação de *strings*.

2) Fazer um programa escrito na linguagem C que leia o Número do Passaporte em uma string, no formato LLDDDDDD, onde L é uma letra maiúscula e D é um dígito e escreva o passaporte com numeração subsequente. Caso o número for 999999, encontre o conjunto de letras subsequente.

String lida	Saída Gerada
AB123456	AB123457
AB123459	AB123460
AB123999	AB124000
AB999999	AC000000

AZ999999	BA000000
ZZ999999	AA000000

3) Desenvolva um programa em linguagem C que leia uma matriz de caracteres e uma string representando uma palavra. O programa deverá verificar se a palavra está presente na matriz, considerando que ela pode ser formada horizontalmente ou verticalmente. As dimensões da matriz (número de linhas e colunas) são definidas usando a diretiva `#define`. O programa deve exibir a posição inicial (linha e coluna) onde a palavra começa, ou `-1 -1` caso a palavra não seja encontrada na matriz.

Solução do TDE - Strings

— Leitura Complementar —

O tópico de strings está disponível nos seguintes e-books da UCS:

- Minha Biblioteca
 - DAMAS, Luís. Linguagem C
 - Capítulo 7
 - EDELWEISS, Nina. Algoritmos e programação com exemplos em Pascal e C
 - Capítulo 10
 - MANZANO, José Augusto N. G. Estudo dirigido de linguagem C
 - Seção 6.4
 - MANZANO, José Augusto N. G. Linguagem C acompanhada de uma xícara de café
 - Capítulo 12
 - MANZANO, José Augusto N. G. Programação de computadores com C/C++
 - Seção 5.5
 - PINHEIRO, Francisco de Assis Cartaxo. Elementos de programação em C
 - Capítulo 15
 - SOFFNER, Renato. Algoritmos e programação em linguagem C
 - Seção 5.1
- BV
 - DEITEL, Paul J.; DEITEL, Harvey M.; CAETANO, César Augusto Cardoso. C : como programar
 - Capítulo 8
 - MIZRAHI, Victorine Viviane. Treinamento em linguagem C
 - Capítulo 7
 - Strings (200-201),
 - As funções para manipulação de strings (201-204),
 - Outras funções de manipulação de strings (página 205-210)